

Lambda calculus

Exercise sheet 5

Exercise 5.1. Let $ID = \lambda x.x$ and $\Omega = (\lambda x.xx)(\lambda x.xx)$. Reduce the following terms to normal form:
 (a) $(\lambda xy.yx) F ID$; (b) $(\lambda x.\lambda y.xy) y$; (c) $TRUE ID \Omega$; (d) $\bar{1} \bar{2} \bar{3}$; (e) $SND(FST(PAIR(PAIR x y) z))$.

Exercise 5.2. Define as lambda terms: (a) disjunction; (b) implication; (c) exclusive disjunction.

Exercise 5.3. Define the following functions for Church numerals: (a) zero test: $\bar{0} \mapsto TRUE$, $\bar{0} \neq \bar{n} \mapsto FALSE$; (b) parity test: $\overline{2k} \mapsto TRUE$, $\overline{2k+1} \mapsto FALSE$; (c) exponentiation: $\bar{n}, \bar{m} \mapsto \bar{n}^m$.

Exercise 5.4. Give explicit definitions for addition, multiplication, and exponentiation of Church numerals, without relying on `SUCC`, `ADD`, and `MULT`, respectively.

Exercise 5.5. Define the following functions for pairs:

- (a) `SWAP`, which exchanges a pair's components;
- (b) `MAPPAIR`, which receives f, g and a pair (x, y) , and returns (fx, gy) .

Exercise 5.6. Use pairs to construct predecessor function on Church numerals, with predecessor of zero equal to zero. Then construct truncated subtraction, returning the numeral for $\max(m - n, 0)$ on inputs $\bar{m}, \bar{n}$.

A **Church list** $[a_1, \dots, a_n]$ is represented by the term $\lambda cz.ca_1(ca_2(\dots(ca_n z)\dots))$.

It takes a *fold function* c and the *bottom* z . In particular, the empty list is $NIL = \lambda cz.z$, and a new element (*head*) is appended to a list via the *constructor* $CONS = \lambda hlcz.ch(lcz)$. Here `CONS` is a *constructor*: given an element h and a Church list l , it constructs the list with head h and tail l . For example, $[a, b]$ is represented by $CONS a (CONS b NIL) \rightarrow_{\beta}^* \lambda cz.ca(cbz)$.

Exercise 5.7. Define the functions for Church lists:

- (a) `SUM` and `PROD`, which return the sum resp. the product of elements, for a list of numerals;
- (b) `LEN`, which returns the length as a Church numeral;
- (c) `APP`, which appends one list with another;
- (d) `MAP`, which applies a function to each element of a list;
- (e) `REV`, which reverses a list.